

Session 8 : Intégration de Verlet et Position Based Dynamics

Objectifs de la session

- Comprendre **pourquoi** l'intégration d'Euler explose sur les systèmes de particules (ressorts, tissus).
- Maîtriser l'intégration de **Verlet** : vitesse implicite, stabilité, dérivation de la formule.
- Appliquer un **damping** adapté à Verlet pour dissiper l'énergie.
- Découvrir **Position Based Dynamics** (PBD) : résoudre des contraintes de distance en manipulant directement les positions.
- Assembler ces briques dans un TP : un drapeau animé en tissu.

1. Le Problème : Pourquoi Euler Explose

Rappel : intégration d'Euler explicite

À chaque pas de temps, Euler met à jour position et vitesse séparément :

$$x_{n+1} = x_n + v_n \cdot dt \quad ; \quad v_{n+1} = v_n + a_n \cdot dt$$

C'est simple, mais la position et la vitesse sont **décalées** : on intègre la vitesse avec l'accélération **d'avant** le déplacement.

La loi de Hooke (rappel — la source du problème)

Un ressort exerce une force de rappel proportionnelle à l'étirement :

$$\vec{F} = -k \cdot (L - L_0) \cdot \hat{u}$$

- k : raideur, L : longueur actuelle, L_0 : longueur au repos, $\hat{u}$: direction unitaire.

Pour un tissu rigide, on veut un k **élevé**. Mais avec Euler, un grand k rend le système **instable** : l'énergie augmente à chaque pas jusqu'à l'explosion.

*Force de Hooke — ce qu'on *évitera* dans le TP.*

```
// Force de rappel appliquée à la particule `a` par le ressort (a,b)
function springForce(a, b, k, restLength) {
  const delta = b.clone().sub(a);
  const L = delta.length();
  const u = delta.normalize(); // direction unitaire hat(u)
  return u.multiplyScalar(-k * (L - restLength)); // -k * (L - L0) * u
}
```

Ce code fonctionne, mais pour l'utiliser avec Euler sans explosion, il faudrait un dt minuscule ou un k faible (tissu mou). C'est inacceptable pour un jeu à 60 FPS.

! La solution de cette session

Au lieu de **calculer des forces** (Hooke) puis d'intégrer (Euler), on va :

1. Intégrer avec **Verlet** — une méthode **symplectique** qui ne dérive pas en énergie.
2. Maintenir la rigidité avec **PBD** — on corrige directement les positions, sans aucune force ni raideur k .

Résultat : un tissu stable à n'importe quel dt , sans paramètre k à régler.

2. L'Intégration de Verlet

Idée de Verlet

Verlet ne stocke **pas** la vitesse explicitement. La vitesse est **déduite** de la différence entre la position actuelle et la position précédente :

$$v \approx \frac{x_n - x_{n-1}}{dt}$$

La formule d'intégration devient :

$$x_{n+1} = x_n + (x_n - x_{n-1}) + a \cdot dt^2$$

💡 D'où vient la formule ?

Par développement de Taylor de $x(t)$ autour de t_n :

$$x(t_n + dt) = x_n + v_n \cdot dt + \frac{1}{2}a_n \cdot dt^2 + O(dt^3)$$

$$x(t_n - dt) = x_n - v_n \cdot dt + \frac{1}{2}a_n \cdot dt^2 + O(dt^3)$$

En additionnant les deux, les termes en v_n s'annulent :

$$x(t_n + dt) + x(t_n - dt) = 2x_n + a_n \cdot dt^2 + O(dt^4)$$

D'où la formule de Verlet :

$$x_{n+1} = 2x_n - x_{n-1} + a_n \cdot dt^2$$

qui est équivalente à $x_n + (x_n - x_{n-1}) + a \cdot dt^2$. Notez que l'erreur passe de $O(dt^3)$ (Euler) à $O(dt^4)$ — Verlet est **plus précise**.

Verlet vs Euler

Critère	Euler explicite	Verlet
Vitesse stockée	Oui (v_n)	Non (implicite)
Précision	$O(dt^3)$	$O(dt^4)$
Stabilité énergie	Mauvaise (dérive)	Bonne (symplectique)
Déplacement manuel	Casse la vitesse	Vitesse s'ajuste seule
Coût mémoire	x, v	x, x_{prev}

Table 1: Verlet est plus précise, plus stable, et tolère qu'on déplace les particules à la main — exactement ce dont PBD a besoin.

❗ Pourquoi PBD exige Verlet

En PBD, on **déplace directement** les positions pour satisfaire les contraintes (section 4). Avec Euler, ce déplacement manuel n’affecte pas v — la vitesse devient incohérente avec la position, et le système “explose” à la frame suivante. Avec Verlet, la vitesse étant $x_n - x_{n-1}$, **toute correction de position met à jour automatiquement la vitesse**. C’est le mariage parfait.

Utilisations typiques de Verlet : cordes, tissus, chevelures, tas de billes, ragdolls.

2.A. Implémentation de Verlet

Une particule Verlet possède :

- position : position actuelle x_n .
- prevPosition : position précédente x_{n-1} .
- acceleration : accélération cumulée des forces (gravité, vent, ...).
- pinned : booléen, fixée au décor (mât, clou, ...).

Une étape de Verlet pour une particule. Voici **exactement** la fonction à coller dans le TP (Mission 1).

```
function verletIntegrate(p, dt) {  
    if (p.pinned) return; // fixée : ne bouge pas  
    const temp = p.position.clone();  
    const velocity = p.position.clone().sub(p.prevPosition);  
    p.position  
        .add(velocity.multiplyScalar(DAMPING)) // vitesse implicite + damping  
        .addScaledVector(p.acceleration, dt * dt); // a·dt²  
    p.prevPosition.copy(temp);  
}
```

- temp sauvegarde x_n avant de l’écraser, pour devenir x_{n-1} au prochain pas.
- velocity est $x_n - x_{n-1}$ (sans diviser par dt — le dt est absorbé dans $a \cdot dt^2$).
- DAMPING (section 3) est un coefficient ≤ 1 appliqué à la vitesse.

3. Le Damping (Amortissement)

Force d'amortissement (forme classique)

On ajoute une force opposée à la vitesse pour dissiper l’énergie :

$$\vec{F}_{\text{damping}} = -b \cdot \vec{v}$$

Sans damping, un système isolé oscillerait indéfiniment (aucune perte).

Damping en Verlet — forme implicite

En Verlet, la vitesse est **implicite** : $\vec{v} \approx x_n - x_{n-1}$. Au lieu d’ajouter une force $-b \cdot \vec{v}$ puis de réintégrer, on **multiplie directement** la vitesse par un coefficient $d \leq 1$:

$$\vec{v}_{\text{damped}} = d \cdot (x_n - x_{n-1})$$

- $d = 1$: aucun amortissement (oscillations perpétuelles).
- $d < 1$: une fraction de la vitesse est perdue à chaque frame.
- $d = 0$: la particule n’a plus d’inertie (tombe comme un bloc).

C’est cette forme — **un simple coefficient** — que vous utiliserez dans le TP.

Damping en Verlet.

```
const DAMPING = 0.99; // 1 = aucun amortissement, <1 = dissipation

// Dans la boucle d'intégration :
// velocity = position - prevPosition
// position += velocity * DAMPING // <- amortissement
```

Le `multiplyScalar(DAMPING)` dans `verletIntegrate` (section 2.A) applique exactement cela.

💡 Choix du coefficient

Pour un drapeau, $d \approx 0.99$ donne un flottement naturel (l'énergie du vent compense les pertes). Pour une corde qui doit s'arrêter, $d \approx 0.9$ à 0.95 . Trop bas (< 0.9), le mouvement devient "pâteux" et artificiel.

4. Position Based Dynamics (PBD)

Principe de PBD

Au lieu de calculer des forces complexes (Hooke) et de les intégrer, on **corrige directement les positions** des particules pour qu'elles respectent des contraintes géométriques : distance, angle, volume, etc.

Une **contrainte** est une règle du type " p_1 et p_2 doivent être à distance L_0 ". Si la règle est violée, on déplace les particules pour la restaurer.

Contrainte de distance — la mathématique

Soient deux particules p_1, p_2 à distance $L = \|p_2 - p_1\|$. On veut $L = L_0$. L'écart est $\Delta L = L - L_0$. On corrige chaque particule de la moitié de l'écart, le long de la direction unitaire $\hat{u} = \frac{p_2 - p_1}{L}$:

$$p_1 \rightarrow p_1 + \frac{1}{2}\Delta L \cdot \hat{u} \quad ; \quad p_2 \rightarrow p_2 - \frac{1}{2}\Delta L \cdot \hat{u}$$

Si une seule particule est mobile, elle absorbe **tout** l'écart (ΔL au lieu de $\Delta L/2$). Si les deux sont fixées, on ne corrige rien.

Algorithme d'une frame PBD

1. **Intégration** — appliquer Verlet à toutes les particules (gravité, vent, damping).
2. **Contraintes** — pour chaque contrainte, corriger les positions (fonction `satisfyConstraint`).
3. **Itérations** — répéter l'étape 2 N fois pour converger (voir ci-dessous).
4. **Rendu** — afficher les positions corrigées.

Contrainte de distance PBD. Voici **exactement** la fonction à coller dans le TP (Mission 2).

```
function satisfyConstraint(p1, p2, restLength) {
  const delta = p2.position.clone().sub(p1.position);
  const dist = delta.length();
  if (dist === 0) return;
  const diff = (dist - restLength) / dist; // écart relatif
  const correction = delta.multiplyScalar(0.5 * diff);
  if (!p1.pinned) p1.position.add(correction);
}
```

```
if (!p2.pinned) p2.position.sub(correction);
}
```

- delta est $p_2 - p_1$; dist est L ; diff est $\Delta \frac{L}{L}$ (écart **relatif**).
- correction = delta * 0.5 * diff vaut $\frac{1}{2} \Delta L \cdot \hat{u}$ — exactement la formule ci-dessus.
- Les gardes !pinned gèrent les particules fixées : si une seule est mobile, elle reçoit la correction complète (l'autre add/sub est sautée).

💡 Itérations = raideur (le point clé de PBD)

Une **seule** passe de contraintes ne restaure pas exactement L_0 : corriger une contrainte en déplace les voisines, qui ne sont plus à leur distance de repos. C'est un système **couplé**.

La solution : **itérer**. En répétant satisfyConstraint N fois par frame, les corrections se propagent et le système converge vers une solution où **toutes** les contraintes sont proches de L_0 .

- $N = 1$: tissu très élastique, la gravité l'étire.
- $N = 5$: tissu mou mais cohérent.
- $N = 15$ à 20 : tissu rigide, réaliste pour un drapeau.
- $N = 50$: quasi-rigide (corde tendue).

Pas de paramètre k : la raideur émerge du nombre d'itérations. C'est ce qui rend PBD si robuste — et si utilisé dans les moteurs modernes (PhysX, Bullet, Havok).

💡 Pourquoi PBD est inconditionnellement stable

On ne fait que **déplacer** des positions d'une quantité bornée ($\Delta \frac{L}{2}$). Il n'y a aucune force qui puisse diverger, aucun k trop grand, aucun dt critique. Le pire cas (N faible) donne un tissu mou — jamais une explosion. À comparer avec Hooke + Euler, où un k un peu trop grand fait tout sauter.

5. TP : Drapeau en Tissu (Verlet + PBD)

💡 Objectif du TP

Animer un drapeau accroché à un mât. Le tissu est une grille de particules liées par des contraintes de distance. Le fichier de départ session08_cloth.html fournit la scène Three.js, le mât, la géométrie du drapeau, le vent et la GUI — il ne reste qu'à compléter **deux fonctions** en copiant-collant le code des sections 2.A et 4.

Mise en place

- Ouvrir session08_cloth.html dans le navigateur (via Live Server).
- La scène contient un drapeau rouge accroché à un mât vertical.
- Au départ, les deux fonctions sont vides : le drapeau tombe comme un bloc (la gravité est appliquée mais aucune contrainte ne maintient le tissu). C'est normal.
- La GUI propose : intensité du vent, damping, itérations PBD, fixer/libérer la colonne gauche, reset, FPS.

Missions

Mission 1 — `verletIntegrate(p, dt)`

Copier le code de la section 2.A dans le corps de la fonction. Attention : la particule fixée (`p.pinned`) ne doit pas bouger. La vitesse implicite est `position - prevPosition`, multipliée par `DAMPING`, puis on ajoute `acceleration * dt2`.

Mission 2 — `satisfyConstraint(p1, p2, restLength)`

Copier le code de la section 4. Calculer `delta = p2.position - p1.position`, la distance, l'écart relatif, et corriger chaque particule de la moitié — sauf les particules `pinned`, qui ne bougent pas.

Mission 3 — Observer et régler

Une fois les deux fonctions en place :

- **Itérations PBD** à 1 : le drapeau s'étire et se déchire visuellement (contraintes molles).
- **Itérations PBD** à 15–20 : le drapeau reste rigide et ondule avec le vent.
- **Damping** à 1.0 : oscillations perpétuelles. À 0.95 : le drapeau se stabilise.
- **Vent** à 0 : le drapeau pend sous la gravité. À 3 : il flotte horizontalement.
- Décocher **Fixer colonne gauche** : le drapeau se détache du mât et tombe (les contraintes restent, plus rien ne le tient).

Scénarios à observer

- **Drapeau qui flotte** : avec vent ≈ 2 , damping ≈ 0.99 et 15 itérations, le drapeau ondule de façon réaliste. C'est exactement la technique utilisée pour les drapeaux et capes dans les jeux.
- **Tissu qui se déchire** : avec 1 itération, les contraintes ne convergent pas — la gravité étire le tissu. Cela montre **pourquoi** on itère en PBD (section 4).
- **Stabilité** : aucun k à régler, aucune explosion. Verlet + PBD est inconditionnellement stable — c'est toute la motivation de la session (section 1).

⚠ Pièges à éviter

- Ne pas oublier le `if (p.pinned) return;` dans `verletIntegrate` — sinon le mât s'envole avec le drapeau.
- Dans `satisfyConstraint`, tester `!p1.pinned` et `!p2.pinned` **séparément** : si les deux sont fixées, on ne corrige rien ; si une seule est fixée, l'autre absorbe toute la correction (le code de la section 4 gère ce cas).
- Garder le `if (dist === 0) return;` pour éviter une division par zéro lorsque deux particules se superposent.
- Ne pas modifier l'ordre `p1/p2` du `delta` : la correction doit aller dans le bon sens (le code de la section 4 est déjà correct).

Bonus

Ajouter des contraintes **diagonales** dans `initCloth` (en plus des horizontales et verticales) avec `restLength = SPACING * Math.sqrt(2)`. Observer : le tissu résiste mieux au cisaillement et ne se déforme pas en losange. C'est ce qu'on appelle la **shear constraint** dans les simulateurs de tissu.